

Week 3 (uploadRepo in codecontroller.js explained)

For Week 3, we look

at **uploadRepo** in controllers/codecontroller.js. This function is like the **main function** that uses githubService + MongoDB together. Overall idea:

- Take a **GitHub repo URL** from the user
 - **Download** the repo as ZIP
 - **Summarize** all .js files using OpenAI
 - **Save** summaries in MongoDB using the Code model
 - Send back a **success response** with info
-

Step 1: Getting the repo URL from request

Inside uploadRepo, first lines:

- It reads

```
const { repoUrl } = req.body;
```
- That means the client must send JSON like:

```
{ "repoUrl": "https://github.com/owner/repo" }
```

If repoUrl is missing:

- It returns 400 (bad request) with:

```
{ "error": "Repository URL is required" }
```

So this checks: **“did user even give me a repo link?”**

Step 2: Downloading the repository ZIP

Then it uses your Github service:

- ```
const zipPath = await fetchRepo(repoUrl);
```

What this does:

- Calls the `fetchRepo()` function from `services/githubService.js`
- That function:

- talks to GitHub API with Octokit
- downloads the repo as a ZIP file
- saves it in your uploads/ folder
- returns the path like uploads/repo-name.zip

So after this line, you have the **repo stored locally** on the server.

---

### Step 3: Processing the ZIP → getting file summaries

Next line:

- `const processedFiles = await processRepo(zipPath);`

This uses **processRepo()** from `githubService.js`:

- It opens the ZIP with `AdmZip`
- Goes through all entries
- Keeps only `.js` files
- For each `.js`:
- reads the file content
- sends the code to `generateSummary(code)` (OpenAI)
- builds an object `{ fileName, code, summary }`
- Returns an array of all those objects

So now `processedFiles` is an array like:

```
[
 { fileName: 'index.js', code: '...', summary: '...' },
 { fileName: 'App.js', code: '...', summary: '...' },
 ...
]
```

This is basically: “**GitHub repo turned into summarized JS files**”.

---

### Step 4: Saving everything into MongoDB

Now you already have Code model imported at top:

```
const Code = require('../models/code');
```

Then in uploadRepo there is a for loop:

- For each file in processedFiles:
- It does:

```
const newCode = new Code({
 fileName: file.fileName,
 code: file.code,
 summary: file.summary,
});
await newCode.save();
```

This means:

- For every JS file in the repo
- Create a new Mongo document
- Save it in the codes collection

So you end up with **one DB record per JS file** in the repo. Later, other parts of your app (like /api/search) can use these stored docs.

---

## Step 5: Sending response to the client

After it saves all files, it sends back:

```
res.json({
 message: 'Repository processed successfully',
 files: processedFiles,
});
```

So the frontend or whoever called this endpoint will get:

- A message string
- The array of all processed files (same data that got saved in Mongo)

---

## Step 6: Error handling

All this is wrapped in `try { ... } catch (err) { ... }`. If anything fails (bad URL, GitHub issue, OpenAI fail, Mongo fail, etc.):

- It logs the error in server console:
- `console.error('Error in uploadRepo:', err);`
- Sends a 500 response:

```
res.status(500).json({ error: err.message });
```

So the client knows something went wrong.

---

## Simple conclusion

You can think of `uploadRepo` like this:

1. **User**: “Here is my GitHub repo URL.”
2. **uploadRepo**: “Ok, I will download your repo, open it, read all JS files, ask AI to summarize them, store them in my database, and then tell you what I did.”
3. If anything breaks in the middle → Sorry, something broke, here is the error message.”

That’s basically Week 3: understanding how `uploadRepo` is flow.

## Week 4 Implemented the githubService.js

So basically services/githubService.js is the part of your backend that **goes to GitHub, downloads a repo, opens it, takes out .js files, and then sends those files to OpenAI for summary.** It's like bring the code from GitHub into our system It has mainly 2 functions:

- **fetchRepo(repoUrl)**
- **processRepo(zipPath)**

And it also has a helper called **getOctokit()**.

---

### 1) Authentication / Octokit setup (GitHub API client)

GitHub doesn't just allow anyone to download stuff from API without limits, so we use a library called **Octokit** (official GitHub API client).

#### **getOctokit() what it does**

- It creates **one Octokit object** and reuses it.
- Like, first time it will create it, next time it will return the same one.

Why? Because creating it again and again is waste and slow.

#### **Why “dynamic import”**

Inside getOctokit() your code does:

- `await import('@octokit/rest')`

That means it loads Octokit only when needed. It's kind of like “load it on demand”.

#### **Token usage**

It uses:

- `auth: process.env.GITHUB_TOKEN`

So it expects you to have GITHUB\_TOKEN in .env. That token is like a password for GitHub API access.

---

### 2) fetchRepo(repoUrl) — download the repo as a ZIP

This function's job is very simple:**Input:** a repo url  
like https://github.com/owner/repo**Output:** a local file path  
like uploads/repo.zipStep by step:

#### **a) Take owner and repo name from the URL**

The code does something like:

- remove https://github.com/
- then split by /

So if URL is:

- https://github.com/facebook/react

Then:

- owner = facebook
- repo = react

#### **b) Download ZIP using GitHub API**

Then it calls:

- octokit.rest.repos.downloadZipballArchive({ owner, repo, ref: 'main' })

This tells GitHub:

- “Give me the whole repo as zip”
- From branch main

(If the repo uses master not main, then this can fail, just saying)

#### **c) Save ZIP into uploads folder**

It saves it here:

- uploads/<repo>.zip

So example:

- uploads/react.zip

It writes the bytes using fs.writeFileSync(...).

#### **d) Return the zip path**

Finally it returns that file path so next function can use it. So  
basically fetchRepo() is: **GitHub** → **ZIP** → **save to disk**.